Лабораторна робота **№ 1**

З дисципліни **«Розробка Інтернет клієнт-серверних систем»**

Виконав:

Студент 3 курсу

3 групи ІПЗ

Сідун Владислав Владиславович

2026

Тема: Вбудовані типи даних та основні операції в Go

Мета: Ознайомитися з основними способами роботи з вбудованими типами

даних (числові, логічні, рядкові, масиви і т.д.) та основними операціями мови

програмування Go.

Завдання: Навчитися використовувати вбудованих типів даних мови Go,

навчитися виконувати базові операції над ними та застосовувати їх при розробці

простих програм.

Виконання

Варіант 11 : 2,7,8,12,23,26,28,38,40,42, 46,55,58,59,61

```
package main

import (
  "fmt"
  "math"
  "strings"
)

func main() {
  fmt.Println("==================================================
")
  fmt.Println("       ЛАБОРАТОРНА РОБОТА №1 | ВАРІАНТ
11          ")
  fmt.Println("==================================================
")
```

```
Task2()
divider()

Task7()
divider()

Task8()
divider()

Task12()
divider()

Task23()
divider()

Task26()
divider()

Task28()
divider()

Task38()
divider()

Task40()
divider()

Task42()
divider()

Task46()
divider()

TaskMathVar13()
divider()
```

```go
  TaskMathVar16()
  divider()

  TaskMathVar17()
  divider()

  TaskMathVar19()

  fmt.Println("\n==================================================
==")
  fmt.Println("                ВИКОНАННЯ
ЗАВЕРШЕНО                ")
  fmt.Println("==================================================
")
}

func divider() {
  fmt.Println("\n--------------------------------------------------
--")
}

// Завдання 2: Виводить довжину рядка, а також індекс першого та
останнього входження букви.
func Task2() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №2 (Рядки)")
  str := "Programming in Go"
  char := "m"

  fmt.Printf("Вхідні дані:\n - Рядок: '%s'\n - Шукана буква:
'%s'\n", str, char)
  fmt.Printf("Результат:\n")
  fmt.Printf(" - Довжина рядка (у байтах): %d\n", len(str))
  fmt.Printf(" - Перше входження: %d\n", strings.Index(str,
char))
  fmt.Printf(" - Останнє входження: %d\n", strings.LastIndex(str,
char))
}
```

```go
// Завдання 7: Створює новий рядок str3 шляхом додавання str2 в
середину str1.
func Task7() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №7 (Рядки)")
  str1 := "Golang lang"
  str2 := " - is awesome - "

  middle := len(str1) / 2
  str3 := str1[:middle] + str2 + str1[middle:]

  fmt.Printf("Вхідні дані:\n - str1: '%s'\n - str2: '%s'\n",
str1, str2)
  fmt.Printf("Результат (str3):\n - '%s'\n", str3)
}

// Завдання 8: Формує нову змінну шляхом видалення символів із
рядка (з 0 по 3-й).
func Task8() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №8 (Рядки)")
  word := "University"

  result := ""
  if len(word) > 3 {
    result = word[4:]
  } else {
    result = "Помилка: слово занадто коротке"
  }

  fmt.Printf("Вхідне слово: '%s'\n", word)
  fmt.Printf("Результат (без перших 4-х символів): '%s'\n",
result)
}

// Завдання 12: Зріз дійсних чисел. Виводить розмір, значення
першого та останнього елементів.
func Task12() {
```

```go
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №12 (Зрізи)")
  slice := []float64{10.5, -3.2, 8.9, 4.1, 0.0}

  fmt.Printf("Вхідний зріз: %v\n", slice)
  fmt.Printf("Результат:\n")
  fmt.Printf(" - Розмір зрізу: %d\n", len(slice))

  if len(slice) > 0 {
    fmt.Printf(" - Перший елемент: %.2f\n", slice[0])
    fmt.Printf(" - Останній елемент: %.2f\n", slice[len(slice)-
1])
  } else {
    fmt.Println(" - Зріз порожній.")
  }
}

// Завдання 23: Заокруглює всі елементи зрізу до найближчого
максимального цілого.
func Task23() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №23 (Зрізи)")
  floats := []float64{1.2, 2.8, -3.5, 4.0}

  ints := make([]int, len(floats))

  for i, v := range floats {
    ints[i] = int(math.Ceil(v))
  }

  fmt.Printf("Вхідний зріз дійсних чисел: %v\n", floats)
  fmt.Printf("Результат (новий зріз цілих чисел): %v\n", ints)
}

// Завдання 26: Видаляє із зрізу всі елементи, які дорівнюють A,
і знаходить суму решти.
func Task26() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №26 (Зрізи)")
  slice := []int{5, 10, 15, 5, 20}
```

```go
  A := 5
  var newSlice []int
  sum := 0

  for _, v := range slice {
    if v != A {
      newSlice = append(newSlice, v)
      sum += v
    }
  }

  fmt.Printf("Вхідні дані:\n - Зріз: %v\n - Число A: %d\n",
slice, A)
  fmt.Printf("Результат:\n - Зріз після видалення: %v\n - Сума
решти: %d\n", newSlice, sum)
}

// Завдання 28: Видаляє із зрізу елементи, які лежать в діапазоні
від індексу 'a' по 'b'.
func Task28() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №28 (Зрізи)")
  slice := []string{"A", "B", "C", "D", "E", "F"}
  a, b := 1, 3

  fmt.Printf("Вхідні дані:\n - Зріз: %v\n - Видалити від індексу
%d по %d\n", slice, a, b)

  if a >= 0 && b < len(slice) && a <= b {
    newSlice := make([]string, 0, len(slice)-(b-a+1))
    newSlice = append(newSlice, slice[:a]...)
    newSlice = append(newSlice, slice[b+1:]...)

    fmt.Printf("Результат:\n - Новий зріз: %v\n", newSlice)
  } else {
    fmt.Println("Результат:\n - Помилка: Некоректні індекси.")
  }
}
```

```go
// Завдання 38: Об'єднує два об'єкти map і виводить результат та кількість елементів.
func Task38() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №38 (Карти/Maps)")
  map1 := map[int]int{1: 100, 2: 200}
  map2 := map[int]int{3: 300, 2: 999}

  merged := make(map[int]int)
  for k, v := range map1 {
    merged[k] = v
  }
  for k, v := range map2 {
    merged[k] = v
  }

  fmt.Printf("Вхідні дані:\n - Map 1: %v\n - Map 2: %v\n", map1, map2)
  fmt.Printf("Результат:\n - Об'єднана Map: %v\n - Кількість елементів: %d\n", merged, len(merged))
}

// Завдання 40: Видаляє всі елементи, значення яких закінчуються підрядком A.
func Task40() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №40 (Карти/Maps)")
  myMap := map[int]string{1: "apple_tree", 2: "banana", 3: "red_apple"}
  A := "apple"

  fmt.Printf("Вхідні дані:\n - Map: %v\n - Підрядок A: '%s'\n", myMap, A)

  for k, v := range myMap {
    if strings.HasSuffix(v, A) {
      delete(myMap, k)
    }
```

```go
  }

  fmt.Printf("Результат:\n - Мар після очищення: %v\n", myMap)
}

// Завдання 42: Видаляє елементи, значення і ключі яких кратні A,
та знаходить добуток решти.
func Task42() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №42 (Карти/Maps)")
  myMap := map[int]int{2: 4, 3: 9, 4: 8, 5: 10}
  A := 2
  product := 1
  hasElements := false

  fmt.Printf("Вхідні дані:\n - Мар: %v\n - Число A: %d\n", myMap,
A)

  for k, v := range myMap {
    if k%A == 0 && v%A == 0 {
      delete(myMap, k)
    } else {
      product *= k * v
      hasElements = true
    }
  }

  if !hasElements {
    product = 0
  }
  fmt.Printf("Результат:\n - Мар після видалення кратних: %v\n",
myMap)
  fmt.Printf(" - Добуток ключів та значень, що залишилися: %d\n",
product)
}

// Завдання 46: Розраховує результат виразу: (val1 * 3 + val1) /
4 * val2.
```

```go
func Task46() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №46 (Арифметика)")
  val1, val2 := 4.0, 2.0

  result := ((val1*3 + val1) / 4.0) * val2

  fmt.Printf("Вхідні дані: val1 = %.1f, val2 = %.1f\n", val1,
val2)
  fmt.Printf("Результат виразу: %.2f\n", result)
}

// TaskMathVar13: Варіант 13
func TaskMathVar13() {
  fmt.Println("---> ВИКОНАННЯ: Варіант 13")
  n := 2.0

  numerator := 5.0 * n * math.Cos(n)
  denominator := math.Sqrt(math.Pow(n, 3))
  result := numerator / denominator

  fmt.Printf("Вхідне число n: %.2f\n", n)
  fmt.Printf("Результат формули: %.4f\n", result)
}

// TaskMathVar16: Варіант 16
func TaskMathVar16() {
  fmt.Println("---> ВИКОНАННЯ: Варіант 16")
  n := 2.0

  numerator := 3.0*math.Sin(n) - 15.0
  denominator := math.Sqrt(math.Pow(n, 5))
  result := numerator / denominator

  fmt.Printf("Вхідне число n: %.2f\n", n)
  fmt.Printf("Результат формули: %.4f\n", result)
}
```

```go
// TaskMathVar17: Варіант 17
func TaskMathVar17() {
  fmt.Println("---> ВИКОНАННЯ: Варіант 17")
  n := 2.0

  numerator := 10.0 + 2.0*math.Cos(n)
  denominator := 5.0 - math.Sqrt(math.Pow(n, 5))
  result := numerator / denominator

  fmt.Printf("Вхідне число n: %.2f\n", n)
  fmt.Printf("Результат формули: %.4f\n", result)
}

// TaskMathVar19: Варіант 19
func TaskMathVar19() {
  fmt.Println("---> ВИКОНАННЯ: Варіант 19")
  n := 2.0

  numerator := 2.0*math.Pow(n, 2) - 4.0*n + 10.0
  denominator := 2.0 * n
  result := numerator / denominator

  fmt.Printf("Вхідне число n: %.2f\n", n)
  fmt.Printf("Результат формули: %.4f\n", result)
}
```

```
==================================================
       ЛАБОРАТОРНА РОБОТА №1 | ВАРІАНТ 11
==================================================

---> ВИКОНАННЯ ЗАВДАННЯ №2 (Рядки)
Вхідні дані:
 - Рядок: 'Programming in Go'
 - Шукана буква: 'm'
Результат:
 - Довжина рядка: 17
 - Перше входження: 6
 - Останнє входження: 7

====================================

---> ВИКОНАННЯ ЗАВДАННЯ №7 (Рядки)
Вхідні дані:
 - str1: 'Golang lang'
 - str2: ' - is awesome - '
Результат (str3):
 - 'Golan - is awesome - g lang'

====================================

---> ВИКОНАННЯ ЗАВДАННЯ №8 (Рядки)
Вхідне слово: 'University'
Результат (без перших 4-х символів): 'ersity'

====================================

---> ВИКОНАННЯ ЗАВДАННЯ №12 (Зрізи)
Вхідний зріз: [10.5 -3.2 8.9 4.1 0]
Результат:
 - Розмір зрізу: 5
 - Перший елемент: 10.50
 - Останній елемент: 0.00

====================================

---> ВИКОНАННЯ ЗАВДАННЯ №23 (Зрізи)
Вхідний зріз дійсних чисел: [1.2 2.8 -3.5 4]
Результат (новий зріз цілих чисел): [2 3 -3 4]

====================================

---> ВИКОНАННЯ ЗАВДАННЯ №26 (Зрізи)
Вхідні дані:
 - Зріз: [5 10 15 5 20]
 - Число A: 5
Результат:
 - Зріз після видалення: [10 15 20]
```

```
 - Сума решти: 45

====================================

---> ВИКОНАННЯ ЗАВДАННЯ №28 (Зрізи)
Вхідні дані:
 - Зріз: [A B C D E F]
 - Видалити від індексу 1 по 3
Результат:
 - Новий зріз: [A E F]

====================================

---> ВИКОНАННЯ ЗАВДАННЯ №38 (Карти/Maps)
Вхідні дані:
 - Мар 1: map[1:100 2:200]
 - Мар 2: map[2:999 3:300]
Результат:
 - Об'єднана Мар: map[1:100 2:999 3:300]
 - Кількість елементів: 3

====================================

---> ВИКОНАННЯ ЗАВДАННЯ №40 (Карти/Maps)
Вхідні дані:
 - Мар: map[1:apple_tree 2:banana 3:red_apple]
 - Підрядок A: 'apple'
Результат:
 - Мар після очищення: map[1:apple_tree 2:banana]

====================================

---> ВИКОНАННЯ ЗАВДАННЯ №42 (Карти/Maps)
Вхідні дані:
 - Мар: map[2:4 3:9 4:8 5:10]
 - Число A: 2
Результат:
 - Мар після видалення кратних: map[3:9 5:10]
 - Добуток ключів та значень, що залишилися: 1350

====================================

---> ВИКОНАННЯ ЗАВДАННЯ №46 (Арифметика)
Вхідні дані: val1 = 4.0, val2 = 2.0
Результат виразу: 8.00

---> ВИКОНАННЯ: Варіант 13
Input n: 2.00
Result: -1.4713

====================================
```

```
---> ВИКОНАННЯ: Варіант 16
Input n: 2.00
Result: -2.1694

====================================

---> ВИКОНАННЯ: Варіант 17
Input n: 2.00
Result: -13.9570

====================================

---> ВИКОНАННЯ: Варіант 19
Input n: 2.00
Result: 2.5000

=====================================================
                ВИКОНАННЯ ЗАВЕРШЕНО
=====================================================
```

Відповіді на контрольні запитання

1. Що означає поняття статичної типізації в мові програмування Go? Мова програмування Go є статично типізованою, що означає, що тип кожної змінної визначається під час компіляції програми. Змінна не може змінювати свій тип під час виконання програми. Компілятор перевіряє коректність використання типів до запуску , що забезпечує підвищену безпеку коду та зменшує кількість помилок.

2. Чим відрізняються типи float32 і float64? У яких випадках доцільно використовувати кожен з них? Go підтримує два типи чисел з плаваючою комою: float32 та float64. Основна відмінність полягає в точності та об'ємі пам'яті. float32 займає 32 біти, що створює високий ризик втрати точності (похибки заокруглення)

при складних математичних обчисленнях. float64 займає 64 біти і є стандартом за замовчуванням у Go для забезпечення високої точності обчислень.

3. Які основні категорії вбудованих типів даних існують у Go? Основні категорії: цілі числа , дійсні числа , логічний тип та рядки.

4. У чому відмінність між типами int та uint? Тип int — це знакове ціле число, яке може набувати як додатних, так і від'ємних значень. Тип uint — це беззнакове ціле число, яке може містити виключно невід'ємні (додатні) значення.

5. Які особливості має тип string у Go? Що означає незмінність (immutable) рядків? Тип string представляє послідовність байтів у кодуванні UTF-8. Незмінність (immutability) означає, що після створення рядка його вміст неможливо змінити. *Мінус/Ризик:* Спроба змінити окремий символ рядка за індексом (наприклад, str[0] = 'a') призведе до помилки компіляції. Для зміни тексту необхідно створювати новий рядок.

6. У чому різниця між оголошенням змінної через var та використанням оператора :=? Ключове слово var використовується для повної форми оголошення (наприклад, var x int = 10) або з автоматичним визначенням типу (var x = 10). Оператор := — це коротке оголошення. *Критичне обмеження:* Оператор := дозволений лише всередині функцій, на рівні пакета (глобальні змінні) його використовувати не можна.

7. Чому в Go відсутнє неявне приведення типів? Як правильно виконати приведення типів? У Go відсутнє неявне приведення типів (наприклад, var b float64 = a, де a типу int, є помилкою). Це зроблено для уникнення прихованих помилок втрати даних. Правильний варіант — явне приведення типів, використовуючи тип як функцію: var b float64 = float64(a).

8. Який результат буде отримано при діленні двох змінних типу int? Поясніть чому.

При діленні двох змінних типу int результат також буде типу int. Дробова частина відкидається (відбувається цілочисельне ділення). Наприклад, 5 / 2 дасть результат 2. Для отримання точного результату з дробовою частиною змінні необхідно попередньо привести до типу float64.

9. Які логічні операції підтримуються в Go та як вони працюють? Go підтримує три основні логічні операції: логічне І (&&), логічне АБО (||) та логічне НІ (!). Тип bool може набувати лише значень true або false.

10. Які типові помилки можуть виникнути при роботі з числовими типами малого розміру (int8, uint8)? Тип int8 (8-бітове знакове) та uint8 мають жорсткі обмеження по пам'яті. Головний ризик — переповнення (overflow). Наприклад, максимальне

значення int8 дорівнює 127. Якщо додати до нього 1, значення циклічно перейде в -128, що зламає логіку програми без екстреної зупинки (panic).